Software Testing and Automation

Unit-5 Notes

Web Automation

Selenium Automation Framework

Selenium Framework is a suite of automation testing tools based on the JavaScript framework.

- It could run the tests directly on the target browser, drive the interactions on the required web page and rerun them without any manual input.
- It eliminates repetitive manual testing that consumes lots of time and effort.
- Conforms with the idea of Agile and DevOps, which endorse the continuous delivery workflow.
  Thus, Selenium remains one of the favorite tools for testing as it meets the requirement of quick and reliable testing, which helps enterprises save time and money on testing.

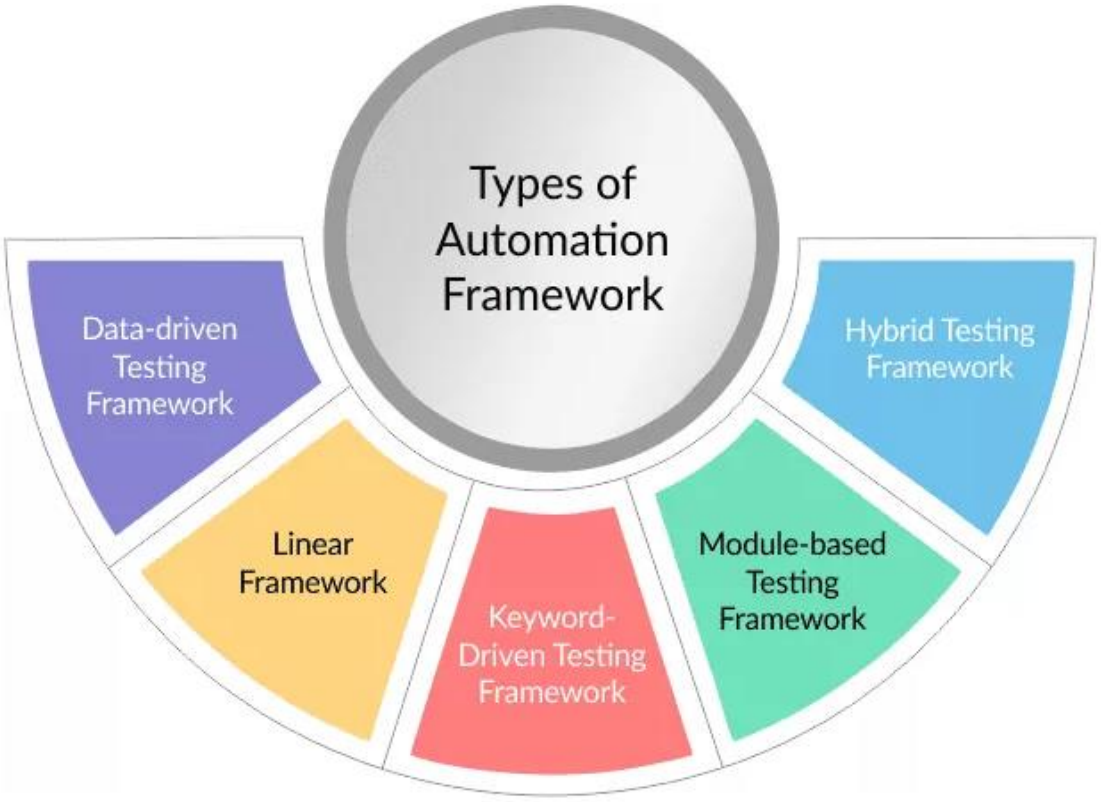
- Framework is a set of rules or best practices that are followed to achieve desired results.
- Selenium Framework is a suite of automation testing tools based on the JavaScript framework.
- It could run the tests directly on the target browser, drive the interactions on the required web page and rerun them without any manual input.
- It eliminates repetitive manual testing that consumes lots of time and effort.
- Conforms with the idea of Agile and DevOps, which endorse the continuous delivery workflow.

Why do we need the Selenium Framework?

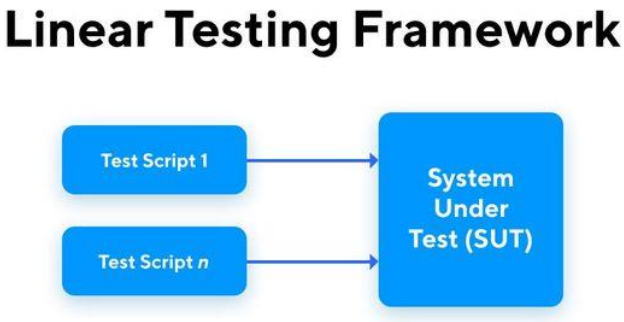
- Easy code maintenance
- Increase in code re-usage
- Higher code readability
- Reduced script maintenance cost

- Reduced tests' time execution
- Reduced human resources
- Easy reporting

Types of Automation Framework



Linear Test Automation Framework



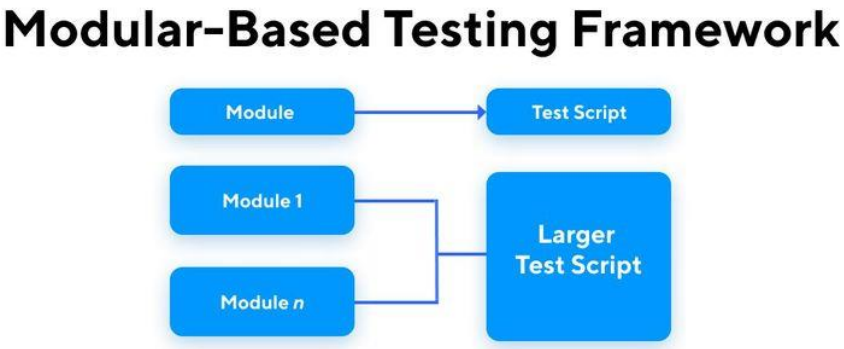
A linear or record-and-playback test automation framework is mostly used for projects that have basic testing needs. Test cases and scripts are created by

testers' interactions on the system-under-test; clicking a "Login" button or inputting usernames and passwords.

However, though the linear test automation framework offers a much faster and low-code approach to design tests, the maintenance effort is also high. Record-and-playback tests break easily. Even the slightest changes in the application or UI code can result in nonreusable tests.

Modular-Based Testing Framework

The name gives it away as the modular-based test frameworks require testers to divide larger test scripts into smaller and manageable units, sections, or functions. Each small module is independently tested with both an incremental and non-incremental approach. These modules are the test cases broken down by the testing framework.
In addition, QA professionals can also store a script in the function library after they are done writing it. It allows them to easily make changes to a certain script rather than accidentally touching other areas, thus reducing risks in testing. Unfortunately, this also asks for good knowledge of the application/software code to find reusable test artifacts and flows.



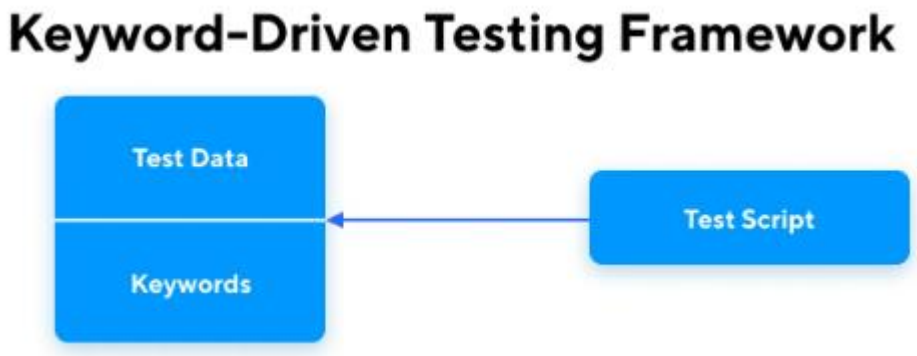
Data-Driven Testing Framework

Excel/CSV, GraphQL, Oracle SQL or databases with JDBC drivers are common datasets used for data-driven testing.

It's quite frequent for testers to run multiple tests on similar features or functions of an application with different test data. Using a data-driven framework allows testers to separate script logic from the test data, preventing them from coding the test data every time into the script itself.

While the linear and modular-based testing framework allows changes to the individual scripts, the data-driven test framework allows testers to store and

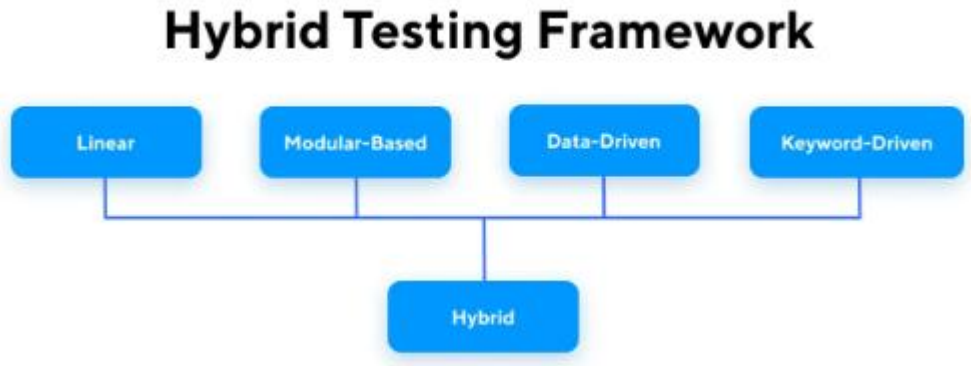
pass data from external sources such as excel sheets, CSV files, text files, ODBC repositories, or SQL tables to test scripts.



The keyword-driven framework lays out a series of predefined actions and methods through keywords. Keywords are a favourite among teams with mixed coding expertise. For example, if a QA team consists of 1 developer and 3 manual testers, the designated developer would be in charge of building those keywords for the manual QAs to write tests.

Keywords in the scripts represent different actions performed for testing an application's GUI. These keywords are sometimes labelled in simple terms such as: "login", or 'click', whereas sometimes they are complex such as 'verifying' or 'click link'.

Hybrid Test Automation Framework



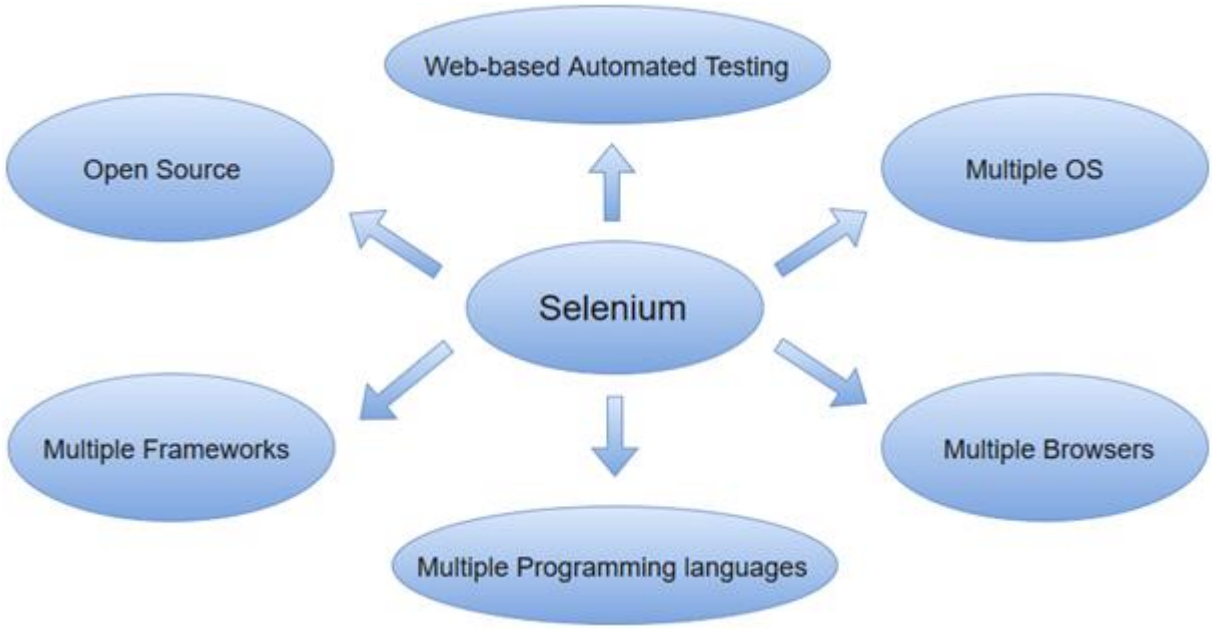
A hybrid test automation framework combines all other test framework types to provide QA teams with a time and cost-efficient testing approach. It mitigates the limitations of each test framework and blends their advantages to improve testing efficiency. As the need for test automation rises, hybrid frameworks successfully address all the issues defined by testers during the testing process.

What is Selenium

Selenium is one of the most widely used open source Web UI (User Interface) automation testing suite.It was originally developed by Jason Huggins in 2004 as an internal tool at Thought Works. Selenium supports automation across different browsers, platforms and programming languages

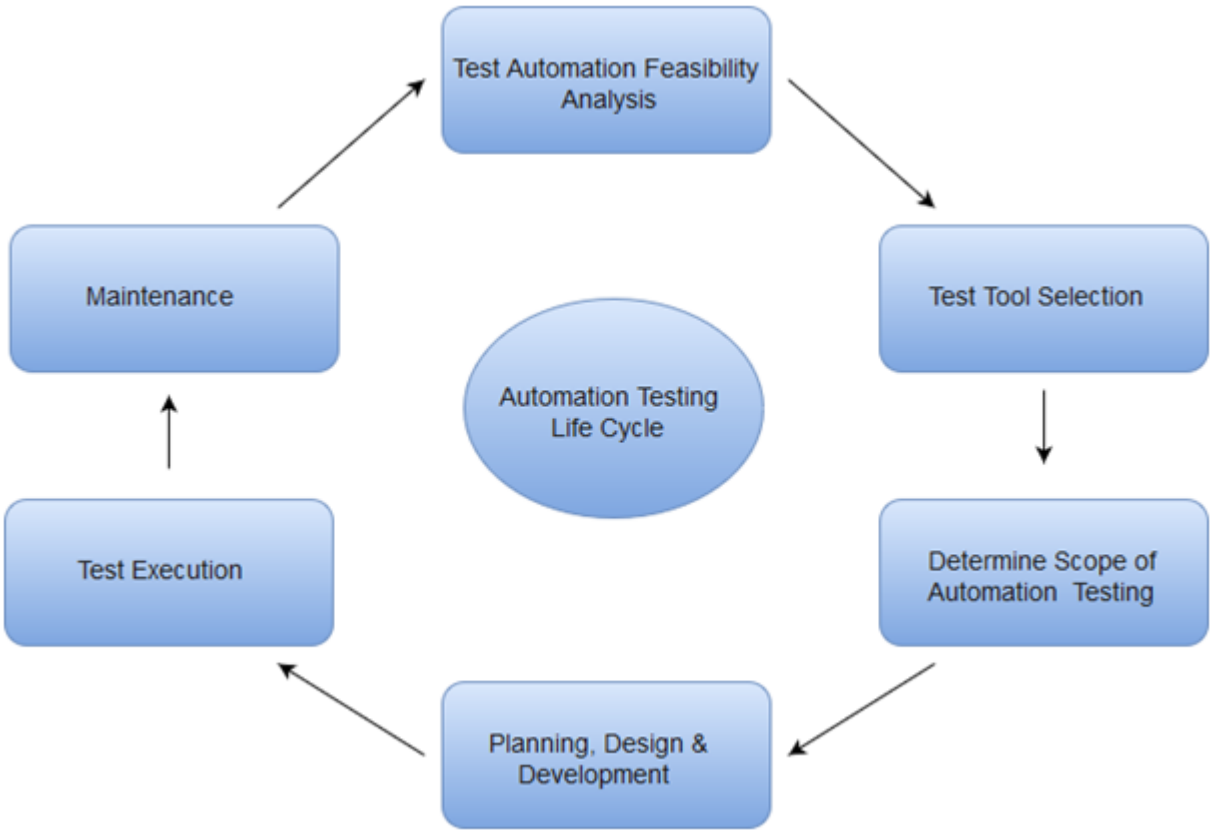
Selenium can be easily deployed on platforms such as Windows, Linux, Solaris and Macintosh. Moreover, it supports OS (Operating System) for mobile applications like iOS, windows mobile and android.

Selenium supports a variety of programming languages through the use of drivers specific to each language. Languages supported by Selenium include C#, Java, Perl, PHP, Python and Ruby. Currently, Selenium Web driver is most popular with Java and C#. Selenium test scripts can be coded in any of the supported programming languages and can be run directly in most modern web browsers. Browsers supported by Selenium include Internet Explorer, Mozilla Firefox, Google Chrome and Safari.



Selenium can be used to automate functional tests and can be integrated with automation test tools such as **Maven**, **Jenkins**, **& Docker** to achieve continuous testing. It can also be integrated with tools such as **TestNG**, & **JUnit** for managing test cases and generating reports.

Automation Testing Life Cycle



Test automation feasibility analysis

Test automation starts with feasibility analysis for any project, it includes activities like identifying the list of software automation testing tools and analyse which all can automate the client's application, support and budget.

- Feasibility analysis can be best done by navigating through all the screens of application under test and list all the unique possible UI components of the application. Evaluate the % of UI components which can be automated by each automation testing tool, here we need to understand that at times we may not be able to automate every UI component present in application under test.

Test tool selection

Identifying the right automation testing tool is critical for the automation testing life cycle since automation testing is highly dependent on the tool used. The developers need to consider budgetary constraints, the familiarity of the team with the automation tool used, along resources available with the team. The choice of the automation tool also depends upon the flexibility and intuitiveness of the team using the tool. The test engineer should define and evaluate the criteria for a pilot test for the automation tool. Testing personnel should then evaluate it based on the different criteria stipulated by the test engineer.

Determine scope of Automation testing

In this step, the testing team identifies the viability of the automation testing. Feasibility analysis is essential for every stage to examine their workability and helps the testing team design the test scripts. Things to be considered in this stage are:

– Deciding which modules of the application should be automated and which ones shouldn’t.

– What test cases can be or need to be automated?

– Understand how to automate those test cases.

– Choose the right tools for automation considering its suitability with the testing goals.

– Analyze the budgets, the cost of implementation, available resources, and available skillset.

Planning, Design and Development

The primary step in this phase of ATLC is to decide which test automation framework to work on. While selecting a suitable tool for your project, you have to keep in mind the technologies required by your software project. So, it is important to perform an in-depth analysis of the product.

While performing automation test planning, testers set up the standards and guidelines for the test procedure creation, hardware, software, and network requirements for the test environment, test data prerequisites, test schedule, error tracking mechanism and the tool, etc. Testers are also responsible for deciding the test architecture, structure of the test program, and test procedure management.

Test Execution

Test execution is the process of executing the code and comparing the expected and actual results. Following factors are to be considered for a test execution process:

- Based on a risk, select a subset of test suite to be executed for this cycle.
- Assign the test cases in each test suite to testers for execution.
- Execute tests, report bugs, and capture test status continuously.
- Resolve blocking issues as they arise.
- Report status, adjust assignments, and reconsider plans and priorities daily.
- Report test cycle findings and status.

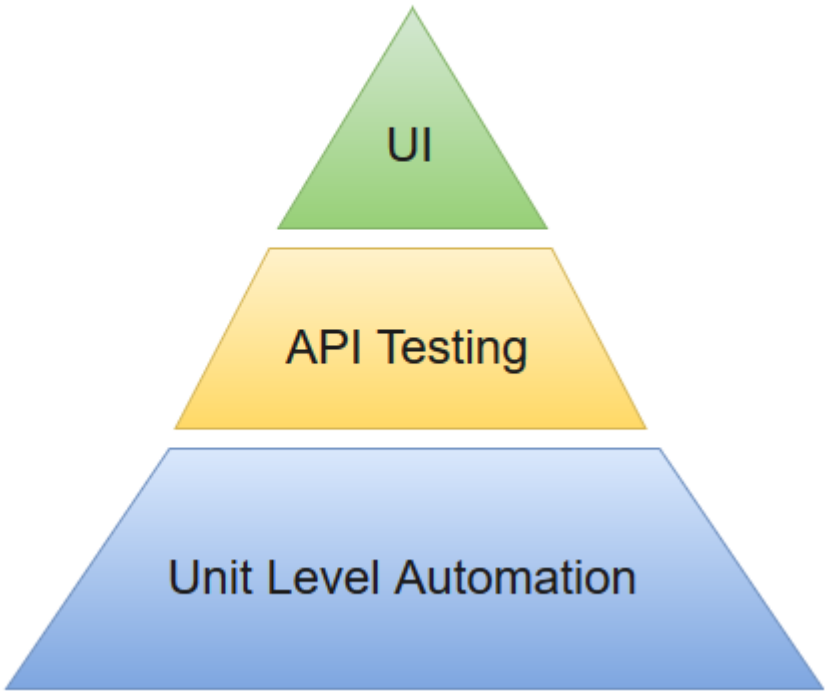
Maintenance

Test maintenance is the process of repairing tests so they stay up to date with code changes. It is also important to update your automation framework infrastructure if there are any changes to the tools or third-party libraries that you use.

Test Automation for Web Applications

The most effective manner to carry out test automation for web application is to adopt a pyramid testing strategy. This pyramid testing strategy includes automation tests at three different levels. Unit testing represents the base and biggest percentage of this test automation pyramid. Next comes, service layer, or API testing. And finally, GUI tests sit at the top. The pyramid looks something like this:

Test Automation Pyramid :



Selenium Features

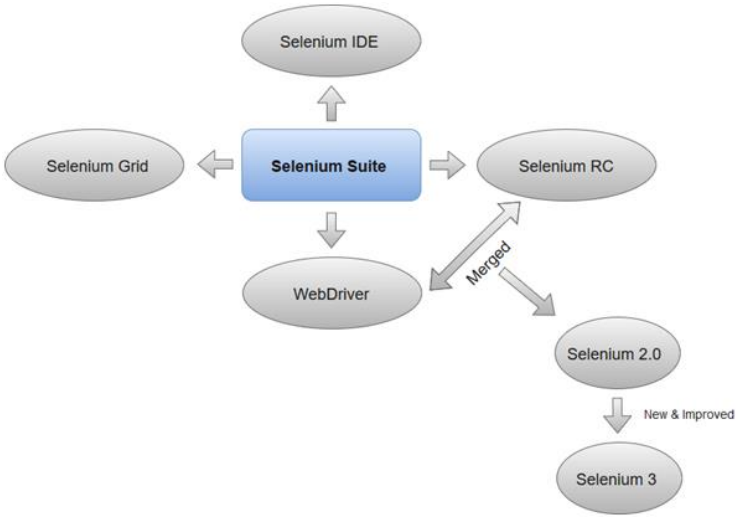
- Selenium is an open source and portable Web testing Framework.
- Selenium IDE provides a playback and record feature for authoring tests without the need to learn a test scripting language.
- It can be considered as the leading cloud-based testing platform which helps testers to record their actions and export them as a reusable script with a simple-to-understand and easy-to-use interface.
- Selenium supports various operating systems, browsers and programming languages. Following is the list:
    - Programming Languages: C#, Java, Python, PHP, Ruby, Perl, and JavaScript
    - Operating Systems: Android, iOS, Windows, Linux, Mac, Solaris.
    - Browsers: Google Chrome, Mozilla Firefox, Internet Explorer, Edge, Opera, Safari, etc.
- It also supports parallel test execution which reduces time and increases the efficiency of tests.
- Selenium can be integrated with frameworks like Ant and Maven for source code compilation.

- Selenium can also be integrated with testing frameworks like TestNG for application testing and generating reports.
- Selenium requires fewer resources as compared to other automation test tools.
- WebDriver API has been indulged in selenium whichis one of the most important modifications done to selenium.
- Selenium web driver does not require server installation, test scripts interact directly with the browser.
- Selenium commands are categorized in terms of different classes which make it easier to understand and implement.
- Selenium Remote Control (RC) in conjunction with WebDriver API is known as Selenium 2.0. This version was built to support the vibrant web pages and Ajax.

Selenium Tool Suite

Selenium is not just a single tool but a suite of software, each with a different approach to support automation testing. It comprises of four major components which include:

1. Selenium Integrated Development Environment (IDE)
2. Selenium Remote Control (Now Deprecated)
3. WebDriver
4. Selenium Grid



1.Selenium Integrated Development Environment (IDE)

Selenium IDE is implemented as Firefox extension which provides record and playback functionality on test scripts. It allows testers to export recorded scripts in many languages like HTML, Java, Ruby, RSpec, Python, C#, JUnit and TestNG. You can use these exported script in Selenium RC or Webdriver.

Selenium IDE has limited scope and the generated test scripts are not very robust and portable.

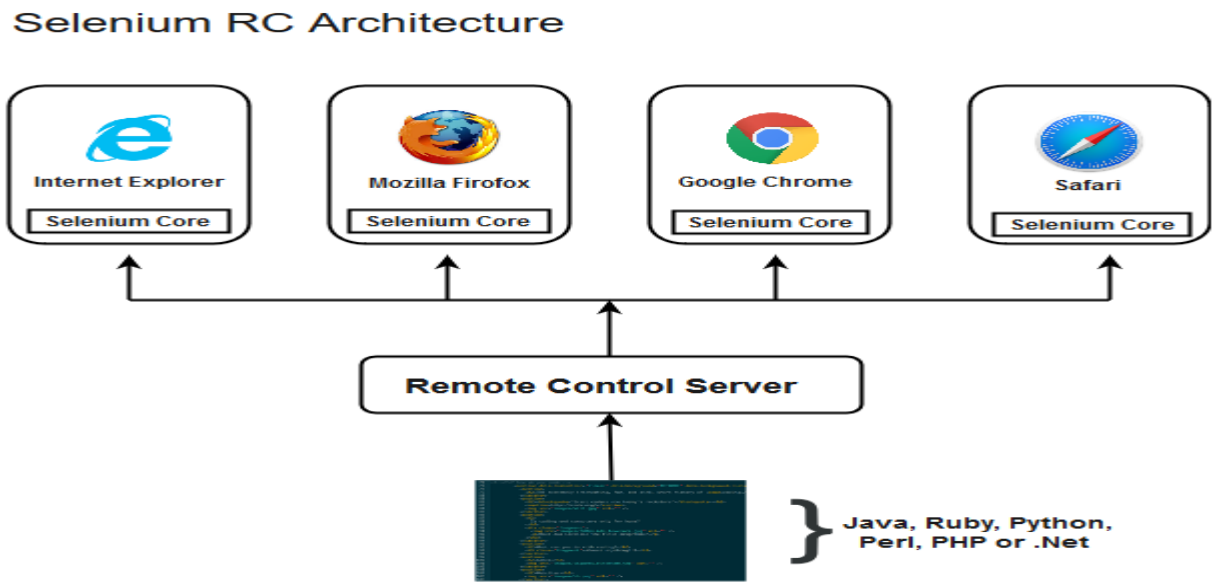
2. Selenium Remote Control

Selenium RC (officially deprecated by selenium)allows testers to write automated web application UI test in any of the supported programming languages. It also involves an HTTP proxy server which enables the browser to believe that the web application being tested comes from the domain provided by proxy server.

Selenium RC comes with two components.

1. Selenium RC Server (acts as a HTTP proxy for web requests).
2. Selenium RC Client (library containing your programming language code).

The figure given below shows the architectural representation of Selenium RC.



Selenium RC had been considered quite effective for testing complex AJAX-based web user interfaces under a Continuous Integration System.

3. Selenium WebDriver

Selenium WebDriver (Selenium 2) is the successor to Selenium RC and is by far the most important component of Selenium Suite. Selenium WebDriver provides a programming interface to create and execute test cases. Test scripts are written in order to identify web elements on web pages and then desired actions are performed on those elements.

Selenium WebDriver performs much faster as compared to Selenium RC because it makes direct calls to the web browsers. RC on the other hand needs an RC server to interact with the web browser.

Since, WebDriver directly calls the methods of different browsers hence we have separate driver for each browser. Some of the most widely used web drivers include:

- Mozilla Firefox Driver (Gecko Driver)
- Google Chrome Driver
- Internet Explorer Driver
- Opera Driver
- Safari Driver
- HTML Unit Driver (a special headless driver)

4. Selenium Grid

Selenium Grid is also an important component of Selenium Suite which allows us to run our tests on different machines against different browsers in parallel. In simple words, we can run our tests simultaneously on different machines running different browsers and operating systems.

Selenium Grid follows the **Hub-Node Architecture** to achieve parallel execution of test scripts. The Hub is considered as master of the network and the other will be the nodes. Hub controls the execution of test scripts on various nodes of the network.

Selenium Waits

The wait commands are essential when it comes to executing Selenium tests. They help to observe and troubleshoot issues that may occur due to variation in time lag.
While running Selenium tests, it is common for testers to get the message "***Element Not Visible Exception***". This appears when a particular web element

with which WebDriver has to interact, is delayed in its loading. To prevent this Exception, Selenium Wait Commands must be used.
In automation testing, Selenium WebDriver wait commands direct test execution to pause for a certain length of time before moving onto the next step. This enables WebDriver to check if one or more web elements are present/visible/enriched/clickable, etc.

Types of Waits in Selenium

Implicit Wait directs the Selenium WebDriver to wait for a certain measure of time before throwing an exception. Once this time is set, WebDriver will wait for the element before the exception occurs.
Once the command is run, Implicit Wait remains for the entire duration for which the browser is open. It's default setting is 0, and the specific wait time needs to be set by the following protocol.
To add implicit waits in test scripts, import the following **package**

```
import java.util.concurrent.TimeUnit;
```

Implicit Wait Syntax

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

Example of Implicit Wait Command

```
Package waitExample;



import java.util.concurrent.TimeUnit;

import org.openqa.selenium.*;

import org.openqa.selenium.firefox.FirefoxDriver;

import org.testng.annotations.AfterMethod;

import org.testng.annotations.BeforeMethod;

import org.testng.annotations.Test;

public class WaitTest {

private WebDriver driver;

private String baseUrl;

private WebElement element;

@BeforeMethod

public void setUp() throws Exception {

driver = new FirefoxDriver();

baseUrl = "http://www.google.com";

driver.manage().timeouts().implicitlyWait(30, TimeUnit.SECONDS);

}
```

```
@Test

public void testUntitled() throws Exception {

driver.get(baseUrl);

element = driver.findElement(By.id("lst-ib"));

element.sendKeys("Selenium WebDriver Interview questions");

element.sendKeys(Keys.RETURN);

List<WebElement> list = driver.findElements(By.className("_Rm"));

System.out.println(list.size());



}

@AfterMethod

public void tearDown() throws Exception {

driver.quit();

}

}
```

Explicit Wait in Selenium

By using the **Explicit Wait** command, the WebDriver is directed to wait until a certain condition occurs before proceeding with executing the code.

Setting Explicit Wait is important in cases where there are certain elements that naturally take more time to load. If one sets an implicit wait command, then the browser will wait for the same time frame before loading every web element. This causes an unnecessary delay in executing the test script.

Explicit wait is more intelligent, but can only be applied for specified elements. However, it is an improvement on implicit wait since it allows the program to pause for dynamically loaded Ajax elements.

In order to declare explicit wait, one has to use **ExpectedConditions.** The following Expected Conditions can be used in Explicit Wait.

1. alertIsPresent()
2. elementSelectionStateToBe()
3. elementToBeClickable()
4. elementToBeSelected()
5. frameToBeAvaliableAndSwitchToIt()
6. invisibilityOfTheElementLocated()
7. invisibilityOfElementWithText()
8. presenceOfAllElementsLocatedBy()
9. presenceOfElementLocated()
10. textToBePresentInElement()
11. textToBePresentInElementLocated()
12. textToBePresentInElementValue()
13. titleIs()
14. titleContains()
15. visibilityOf()
16. visibilityOfAllElements()
17. visibilityOfAllElementsLocatedBy()
18. visibilityOfElementLocated()

To use Explicit Wait in test scripts, import the following **packages** into the script

```
import org.openqa.selenium.support.ui.ExpectedConditions

import org.openqa.selenium.support.ui.WebDriverWait
```

Then, Initialize A Wait Object using *WebDriverWait* Class.

```
WebDriverWait wait = new WebDriverWait(driver,30);
```

Example of Explicit Wait Command

```
package waitExample;


import java.util.concurrent.TimeUnit;


import org.openqa.selenium.By;

import org.openqa.selenium.Keys;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.firefox.FirefoxDriver;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;

import org.testng.annotations.AfterMethod;

import org.testng.annotations.BeforeMethod;

import org.testng.annotations.Test;

public class ExpectedConditionExample {

// created reference variable for WebDriver

WebDriver driver;


@BeforeMethod

public void setup() throws InterruptedException {


// initializing driver variable using FirefoxDriver
```

```
driver=new FirefoxDriver();

// launching gmail.com on the browser

driver.get("https://gmail.com");

// maximized the browser window

driver.manage().window().maximize();

driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);

}


@Test

public void test() throws InterruptedException {

// saving the GUI element reference into a "element" variable of WebElement type

WebElement element = driver.findElement(By.id("Email"));

// entering username

element.sendKeys("dummy@gmail.com");

element.sendKeys(Keys.RETURN);

// entering password

driver.findElement(By.id("Passwd")).sendKeys("password");

// clicking signin button

driver.findElement(By.id("signIn")).click();

// explicit wait - to wait for the compose button to be click-able

WebDriverWait wait = new WebDriverWait(driver,30);
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//div[contains(text(),'COMPOSE')]")));

// click on the compose button as soon as the "compose" button is visible

driver.findElement(By.xpath("//div[contains(text(),'COMPOSE')]")).click();

}


@AfterMethod

public void teardown() {

// closes all the browser windows opened by web driver

driver.quit();

}


}
```

Fluent Wait in Selenium

Fluent Wait in Selenium marks the maximum amount of time for Selenium WebDriver to wait for a certain condition (web element) becomes visible. It also defines how frequently WebDriver will check if the condition appears before throwing the "**ElementNotVisibleException**".

To put it simply, Fluent Wait looks for a web element repeatedly at regular intervals until timeout happens or until the object is found.

Fluent Wait commands are most useful when interacting with web elements that can take longer durations to load. This is something that often occurs in Ajax applications.

The differences between implicit and explicit wait are listed below −

	Implicit Wait	Explicit Wait
1	The driver is asked to wait for a specific amount of time for the element to be available on the DOM of the page.	The driver is asked to wait till a certain condition is satisfied.
2	It is a global wait and applied to all elements on the webpage.	It is not a global wait and applied to a particular scenario.
3	It does not require you to meet any condition.	It is required to satisfy a particular condition. Some of the expected conditions include − • title_contains • visibility_of_element_located • presence_of_element_located • title_is • visibility_of • element_to_be_selected
4	Syntax driver.implicitly_wait(2)	Syntax w = WebDriverWait(driver, 7) w.until(expected_conditions.presence_of_element_located((By.ID, "xyz")))
5	It is simple and easy to implement.	It is more complex in implementation compared to implicit wait.
6	It affects the execution speed since each step waits for this wait till it gets the element it is looking for.	It does not affect the execution speed since it is applicable to a particular element of the page.
7	It does not catch performance issues in the application.	It can catch performance issues in the application.

What are Browser Elements?

Browser elements are different components present on web pages. The most common elements you will notice while browsing is:

- Text boxes
- CTA buttons
- Images
- Hyperlinks
- Radio buttons/checkboxes
- Text area/error messages
- Dropdown box/list box/combo box
- Web table/HTML table
- Frame, etc.

Testing these elements essentially means, you have to check whether they are working fine and responding the way you want them to. For example, if you are testing a text box, what would you test it for?

1. Whether you are able to send text or numbers to the text box
2. If you can retrieve the text that has been passed to the text box, etc.

If you are testing an image, you might want to:

- Download the image
- Upload the image
- Click on the image link
- Retrieve the image title, etc.

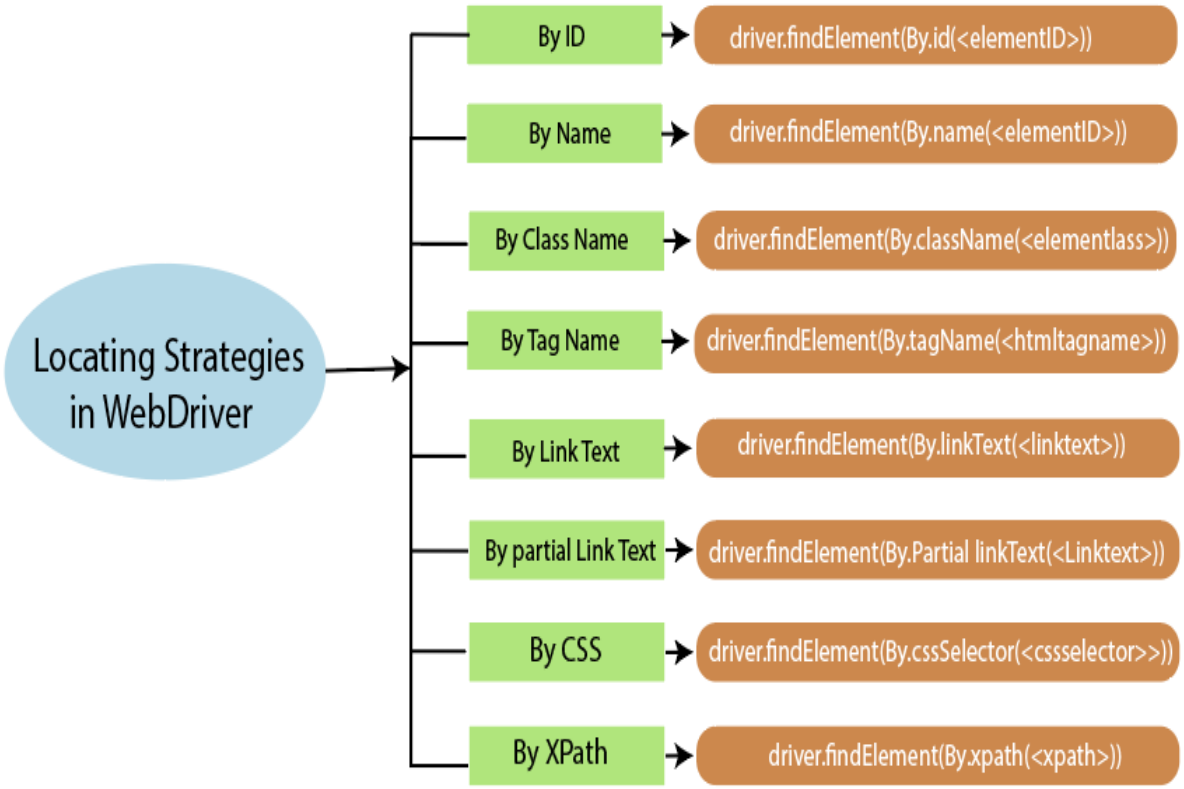
Similarly, operations can be performed on each of the elements mentioned earlier. However, only after the elements are located on the web page, you can perform operations on them and start testing them.

Locating Browser Elements Present on a Web Page

Each element on a web page does have an attribute (property). Elements can have multiple attributes, and most of these attributes will be distinctive for different elements. Consider an example. Suppose there is a page having two elements: an image and a text box.

Both these elements have a 'Name' attribute and an 'ID' attribute. These attribute values need to be distinctive for each of these elements. In other words, no two elements can have the same attribute value.

In the above example, the image and the text box can have neither the same 'ID' nor the same 'Name' value. However, there are some attributes that can be common for a group of elements on the page, e.g., a group of elements can have the same value for 'Class Name'.

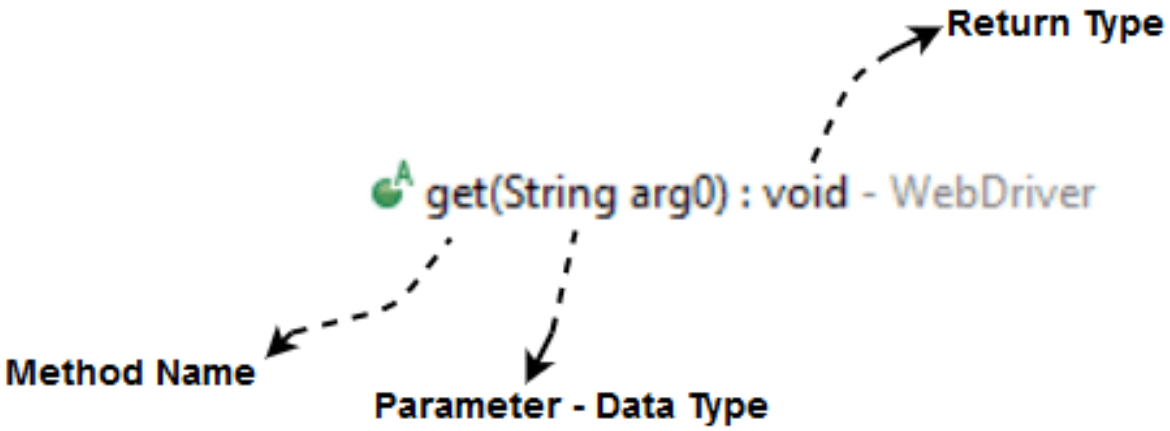


Selenium WebDriver- Commands

- In Selenium WebDriver, we have an entirely different set of commands for performing different operations. Since we are using Selenium WebDriver with Java, commands are simply **methods** written in Java language.
- Every automation Java work file starts with creating a reference of web browser that we wish to use as mentioned in the below syntax.

```
WebDriver driver = new FireFoxDriver
```

To understand the syntax of the methods, consider an example



Method Name: First, we need to create an object of a class to access any method and then all public methods will be visible for the object.

Parameter: An argument that is passed to a method to perform a particular operation.

Return type: Methods can return a value or returning nothing (void). If the void is mentioned after the method, it means, the method is returning no value.

Selenium WebDriver - Browser Commands

- The very basic browser operations of WebDriver include **opening a browser** perform few tasks and then **closing the browser**.
- Given are some of the most commonly used Browser commands for Selenium WebDriver.

1. **Get Command**

Method:

get(String arg0) : **void**

driver.get(URL);

// Or can be written as

String URL = "URL";

driver.get(URL);

2. Get Title Command

Method:

getTitle(): String

In WebDriver, this method fetches the title of the current web page. It accepts no parameter and returns a String.

The respective command to fetch the title of the current page can be written as:

driver.getTitle();

// Or can be written as

String Title = driver.getTitle();

3. Get Current URL Command

Method:

getCurrentUrl(): String

- In WebDriver, this method fetches the string representing the Current URL of the current web page. It accepts nothing as parameter and returns a String value.
- The respective command to fetch the string representing the current URL can be written as:

driver.getCurrentUrl();

//Or can be written as

String CurrentUrl = driver.getCurrentUrl();

4. Get Page Source Command

Method:

getPageSource(): String

- In WebDriver, this method returns the source code of the current web page loaded on the current browser. It accepts nothing as parameter and returns a *String* value.
- The respective command to get the source code of the current web page can be written as:

```
driver.getPageSource();

//Or can be written as

String PageSource = driver.getPageSource();
```

5. Close Command

Method:

close(): **void**

- This method terminates the current browser window operating by WebDriver at the current time. If the current window is the only window operating by WebDriver, it terminates the browser as well. This method accepts nothing as parameter and returns *void*.
- The respective command to terminate the browser window can be written as:

```
driver.close();
```

6. Quit Command

Method:

quit(): **void**

- This method terminates all windows operating by WebDriver. It terminates all tabs as well as the browser itself. It accepts nothing as parameter and returns void.
- The respective command to terminate all windows can be written as:

```
driver.quit();
```

Selenium WebDriver - Navigation Commands

- WebDriver provides some basic Browser Navigation Commands that allows the browser to move backwards or forwards in the browser's history.
- Just like the browser methods provided by WebDriver, we can also access the navigation methods provided by WebDriver by typing driver.navigate() in the Eclipse panel.

1. Navigate To Command

Method:

to(String arg0) : **void**

- In WebDriver, this method loads a new web page in the existing browser window. It accepts *String* as parameter and returns *void*.
- The respective command to load/navigate a new web page can be written as:

```
driver.navigate().to("https://www.youtube.com/");
```

2. Forward Command

Method:

to(String arg0) : **void**

- In WebDriver, this method enables the web browser to click on the **forward** button in the existing browser window. It neither accepts anything nor returns anything.
- The respective command that takes you forward by one page on the browser's history can be written as:

```
driver.navigate().forward();
```

3. Back Command

Method:

back() : **void**

- In WebDriver, this method enables the web browser to click on the **back** button in the existing browser window. It neither accepts anything nor returns anything.

- The respective command that takes you back by one page on the browser's history can be written as:

driver.navigate().back();

4. Refresh Command

Method:

refresh() : **void**

- In WebDriver, this method refresh/reloads the current web page in the existing browser window. It neither accepts anything nor returns anything.
- The respective command that takes you back by one page on the browser's history can be written as:

driver.navigate().refresh();

What is JUnit ?

JUnit is a unit testing framework for Java programming language. It plays a crucial role test-driven development, and is a family of unit testing frameworks collectively known as xUnit.

JUnit promotes the idea of "first testing then coding", which emphasizes on setting up the test data for a piece of code that can be tested first and then implemented. This approach is like "test a little, code a little, test a little, code a little." It increases the productivity of the programmer and the stability of program code, which in turn reduces the stress on the programmer and the time spent on debugging.

Features of JUnit

- JUnit is an open source framework, which is used for writing and running tests.
- Provides annotations to identify test methods.
- Provides assertions for testing expected results.
- Provides test runners for running tests.
- JUnit tests allow you to write codes faster, which increases quality.
- JUnit is elegantly simple. It is less complex and takes less time.
- JUnit tests can be run automatically and they check their own results and provide immediate feedback. There's no need to manually comb through a report of test results.
- JUnit tests can be organized into test suites containing test cases and even other test suites.

- JUnit shows test progress in a bar that is green if the test is running smoothly, and it turns red when a test fails.

What is a Unit Test Case ?

A Unit Test Case is a part of code, which ensures that another part of code (method) works as expected. To achieve the desired results quickly, a test framework is required. JUnit is a perfect unit test framework for Java programming language.

A formal written unit test case is characterized by a known input and an expected output, which is worked out before the test is executed. The known input should test a precondition and the expected output should test a post-condition.

There must be at least two unit test cases for each requirement − one positive test and one negative test. If a requirement has sub-requirements, each sub-requirement must have at least two test cases as positive and negative.

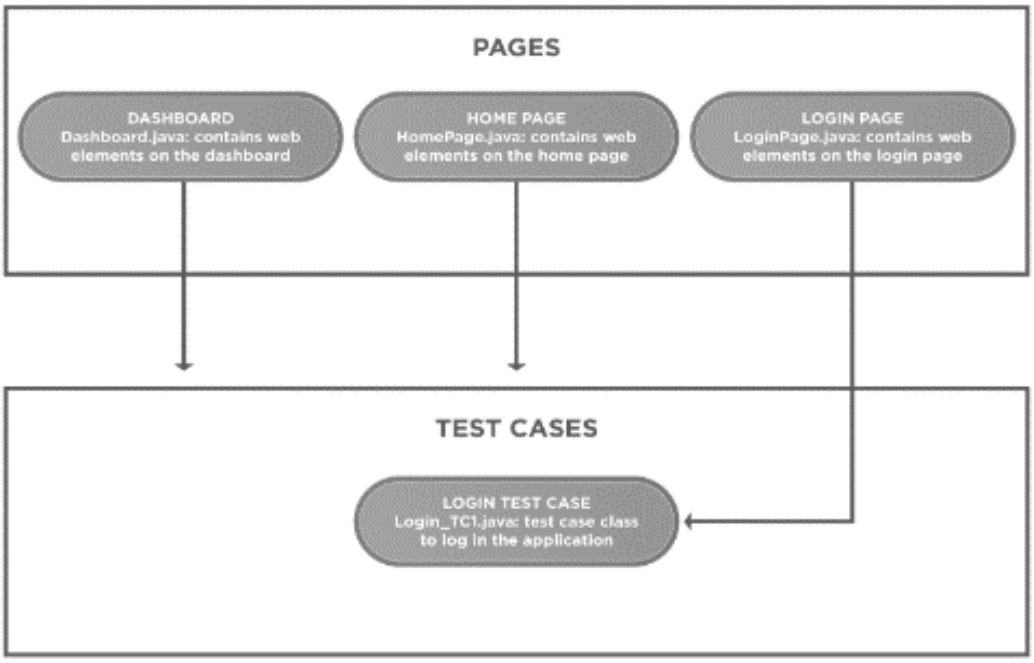
Page Object Concept

What is a Page Object Model in Selenium?

Page Object Model, also known as POM, is a design pattern in Selenium that creates an object repository for storing all web elements. It helps reduce code duplication and improves test case maintenance.

- in the *Page Object Model framework,* we create a class file for each web page.
- This class file consists of different web elements present on the web page. Moreover, the test scripts then use these elements to perform different actions.
- Since each page's web elements are in a separate class file, the code becomes easy to maintain and reduces code duplicity.

The diagram shows a simple project structure implementing *POM*-



Why do we need a Page Object Model?

Selenium automation is not a tedious job. All you need to do is find the elements and perform actions on them. To understand its need, let us consider a simple use case:

- Firstly, navigate to the **Demo website.**
- Secondly, login into the store.
- Thirdly, capture the dashboard title.
- Finally, logout from the store.

Additionally, the code for the use case without using POM would look like as follows:

```
package simple_Project;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

publicclassTest_Without_POM {

public static void main(String[] args) throws InterruptedException {

WebDriverdriver = newChromeDriver();

driver.get("https://www.demoqa.com/books");
```

```
driver.findElement(By.id("login")).click();

driver.findElement(By.id("userName")).sendKeys("gunjankaushik");

driver.findElement(By.id("password")).sendKeys("Password@123");

driver.findElement(By.id("login")).click();

System.out.println("The page title is : "
+driver.findElement(By.xpath("//*[@id=\"app\"]//div[@class=\"main-
header\"]")).getText());

driver.findElement(By.id("submit")).click();

driver.quit();

}

}
```

As visible in the script, we locate each web element, like the *login button, username, password field,* etc. and then perform the corresponding action like *click()* or *sendKeys().* It looks simple, but as the test suite grows, the code becomes complex and challenging to maintain.

A small change in the element will lead to a change in all the scripts that use the web element. It takes a lot of time, and the chances of errors are high.

The resolution to the above problem is by using the *Page Object Model Framework* or *Page Object Model Design Pattern.* Using POM, we will create an object repository, which we can refer to at various points in our test script.

Advantages of Page Object Model

- **Easy Maintenance**

POM is useful when there is a change in a UI element or a change in action.

An example would be: a drop-down menu is changed to a radio button. In this case, POM helps to identify the page or screen to be modified. As every screen will have different Java files, this identification is necessary to make changes in the right files. This makes test cases easy to maintain and reduces errors.

- **Code Reusability**

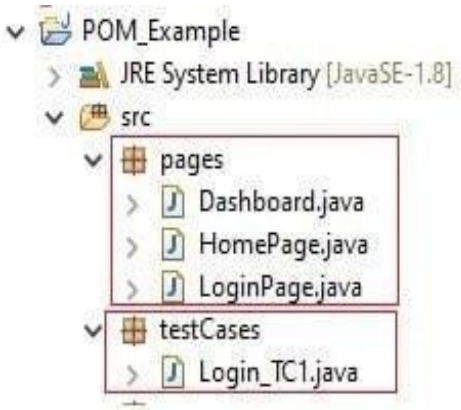
By using POM, one can use the test code for one screen, and reuse it in another test case. There is no need to rewrite code, thus saving time and effort.

- **Readability and Reliability of Scripts**

When all screens have independent java files, one can quickly identify actions performed on a particular screen by navigating through the java file. If a change must be made to a specific code section, it can be efficiently done without affecting other files.

How to implement the Page Object Model in Selenium?

When it comes to the implementation of the *Page Object Model*, consider the use case that we considered previously. The *Page Object Model* project structure would look like below:



Data Transfer Object

DTOs or Data Transfer Objects are objects that carry data between processes in order to reduce the number of methods calls.

The pattern's main purpose is to reduce roundtrips to the server by batching up multiple parameters in a single call. This reduces the network overhead in such remote operations.

Another benefit is the encapsulation of the serialization's logic (the mechanism that translates the object structure and data to a specific format that can be stored and transferred).

It also decouples the domain models from the presentation layer, allowing both to change independently.

How to Use It?

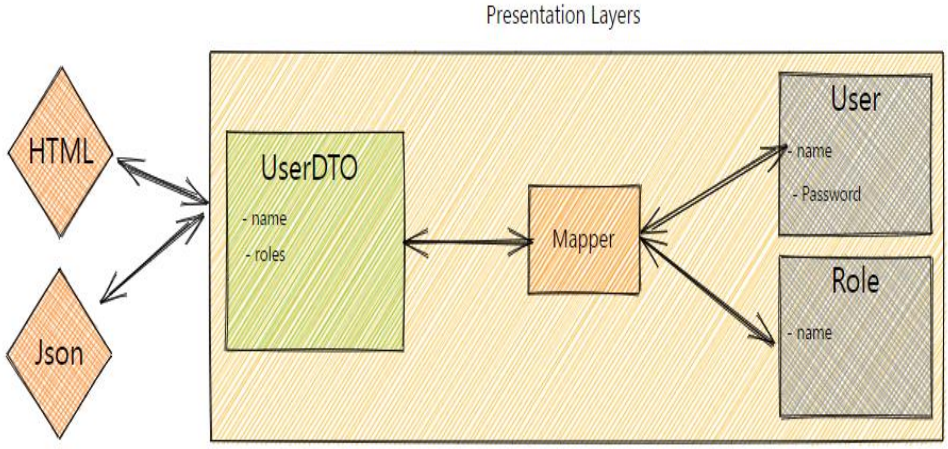
DTOs normally are created as POJOs. They **are flat data structures that contain no business logic.** They only contain storage, accessors and eventually methods related to serialization or parsing.

The data is mapped from the domain models to the DTOs, normally through a mapper component in the presentation or facade layer.

[*POJO, is a straightforward type with no references to any particular frameworks.* ***A POJO has no naming convention*** *for our properties and methods.*]

The data is mapped from the domain models to the DTOs, normally through a mapper component in the presentation or facade layer.

The image below illustrates the interaction between the components:



When to Use It?

DTOs come in handy in systems with remote calls, as they help to reduce the number of them.

DTOs also help when the domain model is composed of many different objects and the presentation model needs all their data at once, or they can even reduce roundtrip between client and server.

With DTOs, we can build different views from our domain models, allowing us to create other representations of the same domain but optimizing them to the clients' needs without affecting our domain design. Such flexibility is a powerful tool to solve complex problems.

Database Connection in Selenium

Selenium Webdriver is limited to Testing user applications using Browser. To use Selenium Webdriver for Database Verification you need to use the JDBC ("Java Database Connectivity").

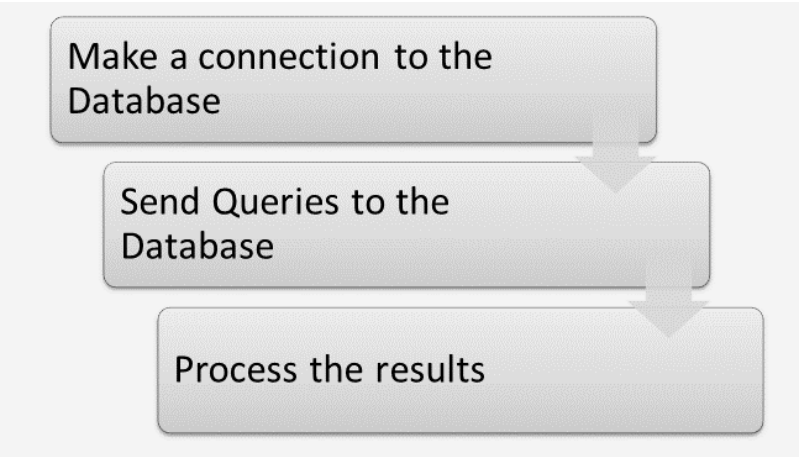
JDBC (Java Database Connectivity) is a SQL level API that allows you to execute SQL statements. The JDBC API provides the following classes and interfaces

- Driver Manager
- Driver

- Connection
- Statement
- ResultSet
- SQLException

How to Connect Database in Selenium

In order to test your Database using Selenium, you need to observe the following 3 steps:



Step 1) Make a connection to the Database

In order to make a connection to the database the syntax is

DriverManager.getConnection(URL, "userid", "password")

Here,

- Userid is the username configured in the database
- Password of the configured user
- URL is of format jdbc:< dbtype>://ipaddress:portnumber/db_name"
- <dbtype>- The driver for the database you are trying to connect.
- To connect to oracle database this value will be "oracle"
- For connecting to database with name "emp" in MYSQL URL will bejdbc:mysql://localhost:3036/emp and the code to create connection looks like

Connection con = DriverManager.getConnection(dbUrl,username,password);

We also need to load the JDBC Driver using the code

Class.forName("com.mysql.jdbc.Driver");

Step 2) Send Queries to the Database

Once connection is made, we need to execute queries.We can use the Statement Object to send queries.

Statement stmt = con.createStatement();

Once the statement object is created use the executeQuery method to execute the SQL queries

stmt.executeQuery(select * from employee;);

Step 3) Process the results

Results from the executed query are stored in the ResultSet Object. Java provides loads of advance methods to process the results. Few of the methods are listed below

Method name	Description
String getString()	Method is used to fetch the string type data from the result set
int getInt()	Method is used to fetch the integer type data from the result set
double getDouble()	Method is used to fetch the double type data from the result set
Date getDate()	Method is used to fetch the Date type object from the result set
boolean next()	Method is used to move to the next record in the result set
boolean previous()	Method is used to move to the previous record in the result set
boolean first()	Method is used to move to the first record in the result set
boolean last()	Method is used to move to the last record in the result set
boolean absolute(int rowNumber)	Method is used to move to the specific record in the result set

Cross Browser Testing

Cross-browser testing, also called browser testing, is a quality assurance (QA) process that checks whether a web-based application, site or page functions as intended for end users across multiple browsers and devices.

Cross-browser testing is a type of compatibility testing, which ensures an application or software integrates as intended with interfacing hardware or software. Compatibility testing can cover hardware, operating systems and devices.

The cross-browser testing process

After the development team builds a web application or site, QA professionals can evaluate the completed project.

A QA professional or team could use these types of environments or visuals, depending on the use case, for cross-browser testing:

- Screenshots, which show how a web application or site displays on a given browser version, but without functionality;
- Simulators, which mimic how an end user would see and interact with the app, without actually testing it on a device;
- Live tests, which either use virtual machines to create a remote computing environment or real devices to replicate the end user experience; and
- Headless browsers, which enable cross-browser testing via a command-line interface, which removes the GUI

Cross-browser testing can be partially automated and incorporated into continuous testing for faster or more accurate results. A failed test creates a feedback loop. Automated cross-browser tests often rely on headless browsers, which enable faster tests as no visuals load.

Cross-browser tests provide information that is gathered together in a feedback loop that developers use to fix broken or otherwise unsatisfactory code. This information might include the platform that caused a failed test, as well as the device and browser version.